
Aplicativo Mobile para Pedidos na Cantina: Promovendo Acessibilidade no Ambiente Acadêmico

1. Introdução

A Faculdade de Tecnologia Arthur de Azevedo, localizada em Mogi Mirim, possui diversos cursos distribuídos em prédios e blocos distintos, o que exige que seus alunos transitem frequentemente entre diferentes áreas do campus para realizar atividades cotidianas. Entre essas atividades, destaca-se o deslocamento até a cantina **Lanche Quentinho**, que, embora centralizada, exige percurso considerável para alunos localizados em blocos mais distantes. Essa situação é agravada para alunos com mobilidade reduzida, seja por deficiência permanente, lesão temporária, limitação de saúde ou qualquer outra condição que torne a locomoção um desafio diário.

Nesse contexto, a tecnologia móvel surge como ferramenta capaz de amenizar essas barreiras arquitetônicas e organizacionais. O presente trabalho propõe o desenvolvimento de um aplicativo *mobile* que permita aos alunos realizarem pedidos antecipados de lanches, bebidas e demais produtos oferecidos pela cantina, com a opção de retirada programada ou entrega em ponto combinado dentro do campus. A proposta busca, essencialmente, democratizar o acesso aos serviços do campus por meio da integração digital.

O artigo está organizado em quatro seções principais. A **Introdução** contextualiza o tema, o problema, a justificativa, os objetivos e a metodologia. A **Revisão Bibliográfica** aprofunda os fundamentos teóricos e tecnológicos que embasam o projeto. A seção **Projeto** descreve a análise de requisitos, as telas e a arquitetura do sistema. Por fim, as **Considerações Finais** retomam os resultados esperados, discutem limitações e propõem trabalhos futuros.

1.1 Tema

Desenvolvimento de um aplicativo *mobile* para realização de pedidos na cantina universitária, com foco na acessibilidade para alunos com mobilidade reduzida.

1.2 Problema

A estrutura física do campus da FATEC Arthur de Azevedo impõe desafios significativos de locomoção para alunos com mobilidade reduzida. A cantina, embora centralizada, exige deslocamentos que podem ser exaustivos ou mesmo inviáveis para estudantes com deficiências motoras, lesões temporárias ou condições de saúde que limitam a capacidade de caminhar longas distâncias. Diante desse cenário, formulam-se as seguintes questões de pesquisa:

P1: Como um aplicativo *mobile* pode melhorar o acesso dos alunos aos serviços da cantina, especialmente para aqueles com mobilidade reduzida?

P2: Quais funcionalidades são essenciais para garantir que o aplicativo atenda às necessidades de todos os usuários?

P3: De que forma a tecnologia pode compensar barreiras arquitetônicas existentes no campus?

Nota didática: A formulação de perguntas de pesquisa é o ponto de partida de todo trabalho acadêmico. Elas orientam a revisão bibliográfica, a coleta de dados e a análise dos resultados. Note que as perguntas acima são específicas, mensuráveis e diretamente relacionadas ao tema proposto.

1.3 Justificativa

A justificativa para o desenvolvimento deste trabalho fundamenta-se em três pilares complementares.

O **pilar social** diz respeito à inclusão e à acessibilidade. Dados do Censo da Educação Superior (INEP, 2023) indicam que cerca de 2,5% dos estudantes universitários brasileiros declaram algum tipo de deficiência, sendo a física a mais recorrente. No entanto, a infraestrutura das instituições de ensino nem sempre acompanha as necessidades desses alunos, gerando barreiras que vão além das arquitetônicas e atingem a participação plena na vida acadêmica. Um

aplicativo que permita pedidos antecipados reduz a necessidade de deslocamentos e contribui para a autonomia desses estudantes.

O **pilar tecnológico** refere-se à onipresença dos dispositivos móveis no cotidiano dos universitários. Segundo a Pesquisa Nacional por Amostra de Domicílios (PNAD, 2022), 98% dos brasileiros com ensino superior possuem smartphone com acesso à internet. Esse cenário torna viável a adoção de soluções baseadas em aplicativos para resolver problemas cotidianos do ambiente acadêmico, sem exigir investimentos adicionais em hardware por parte dos alunos.

O **pilar acadêmico** relaciona-se à aplicação prática dos conhecimentos adquiridos ao longo do curso técnico em informática. O desenvolvimento do aplicativo mobiliza competências em programação *web* (HTML, CSS, JavaScript), banco de dados, experiência do usuário, arquitetura de software e gestão de projetos, configurando-se como um trabalho integrador que consolida a formação do aluno.

As hipóteses que orientam este trabalho são:

H1: A implementação de um aplicativo *mobile* para pedidos antecipados pode reduzir a necessidade de deslocamento dos alunos até a cantina, promovendo maior acessibilidade.

H2: Uma interface simples e intuitiva aumentará a adesão dos alunos ao uso do aplicativo.

H3: O aplicativo pode melhorar a eficiência do serviço da cantina ao permitir o preparo antecipado dos pedidos.

1.4 Objetivo Geral

Desenvolver um aplicativo *mobile* que permita aos alunos da Faculdade de Tecnologia Arthur de Azevedo realizar pedidos de lanches, bebidas e outros produtos oferecidos pela cantina do campus de forma antecipada, reduzindo a necessidade de deslocamento até o local físico, promovendo acessibilidade para alunos com mobilidade reduzida.

1.5 Objetivos Específicos

Para alcançar o objetivo geral, estabelecem-se os seguintes objetivos específicos:

OE1: Levantar e analisar os requisitos funcionais e não funcionais necessários para o desenvolvimento do aplicativo.

OE2: Projetar uma interface intuitiva e acessível, seguindo boas práticas de Experiência do Usuário (UX).

OE3: Implementar um protótipo funcional do aplicativo utilizando tecnologias *web mobile*.

OE4: Realizar testes de usabilidade com alunos do campus para validar a eficácia da solução.

1.6 Metodologia

Este trabalho será desenvolvido por meio de pesquisa aplicada, com o objetivo de criar um aplicativo *mobile* para pedidos na cantina da faculdade. A metodologia estrutura-se nas seguintes etapas:

1. Pesquisa exploratória: levantamento bibliográfico sobre acessibilidade em instituições de ensino, tecnologias *mobile* e experiência do usuário, realizado em bases como Google Acadêmico, Scopus e SciELO. Foram consultados livros, artigos científicos e documentações técnicas oficiais.

2. Pesquisa de campo: aplicação de questionários com alunos da FATEC Arthur de Azevedo para identificar dificuldades de deslocamento, hábitos de consumo na cantina e interesse na solução proposta. A amostra será não probabilística por conveniência, com alunos dos diferentes cursos e períodos.

3. Análise de requisitos: definição das funcionalidades do sistema com base nos dados coletados na pesquisa de campo e na revisão bibliográfica. Os requisitos serão documentados seguindo o padrão IEEE 830-1998.

4. Prototipação: criação de protótipos de baixa fidelidade (*wireframes*) e alta fidelidade (*mockups*) das telas do aplicativo, utilizando ferramentas como Figma ou Adobe XD. Os protótipos serão refinados com base em *feedback* de potenciais usuários.

5. Desenvolvimento: implementação do protótipo funcional utilizando tecnologias *web* (HTML, CSS, JavaScript) com o *framework* React Native, que permite a compilação para Android e iOS a partir de uma única base de código. O banco de dados utilizado será o Firebase Firestore, serviço NoSQL em nuvem do Google.

6. Testes: realização de testes de usabilidade com alunos voluntários para validação da solução. Os testes seguirão o protocolo proposto por Jacob Nielsen, com métricas como taxa de sucesso, tempo de tarefa e satisfação do usuário (SUS — *System Usability Scale*).

"A pesquisa aplicada difere da pesquisa básica por ter como objetivo a solução de um problema concreto e imediato. No âmbito da tecnologia da informação, a pesquisa aplicada resulta frequentemente em artefatos de software que são submetidos a avaliação prática com usuários reais." (WAZLAWICK, 2014, p. 45)

A pesquisa será de natureza qualitativa e quantitativa, combinando análise de dados subjetivos (percepção dos usuários sobre a usabilidade e a acessibilidade) com dados objetivos (tempo de deslocamento economizado, número de pedidos realizados por período, taxa de erro durante a navegação). Essa triangulação metodológica confere maior robustez aos resultados obtidos.

2. Revisão Bibliográfica

A revisão bibliográfica constitui o alicerce teórico sobre o qual se sustenta o desenvolvimento do aplicativo proposto. Esta seção aborda os fundamentos das tecnologias *web*, a distinção entre *front-end* e *back-end*, os conceitos de bancos de dados, os princípios de experiência do usuário, os *frameworks* disponíveis e as questões legais relacionadas à propriedade de *software*.

2.1 Tecnologias utilizadas no desenvolvimento

O desenvolvimento de aplicações *web* modernas apoia-se em três pilares essenciais: a estrutura do documento, a apresentação visual e o comportamento dinâmico. Esses pilares correspondem, respectivamente, às linguagens HTML, CSS e JavaScript, que constituem a tríade fundamental da programação para a internet.

A escolha por tecnologias *web* para o desenvolvimento de um aplicativo *mobile* justifica-se pelo princípio da **progressão responsiva** (*responsive web design*), que permite que uma mesma base de código se adapte a diferentes tamanhos de tela. Conforme destaca Marcotte (2011), o design responsivo elimina a necessidade de manter versões separadas para *desktop* e dispositivos móveis, reduzindo custos de desenvolvimento e manutenção.

Além disso, o uso de tecnologias *web* no contexto *mobile* pode ser feito por meio de abordagens híbridas ou progressivas. As **aplicações web progressivas** (PWAs) combinam o alcance da *web* com as capacidades dos aplicativos nativos, funcionando *offline* parcialmente e podendo ser instaladas na tela inicial do dispositivo (SHEPPARD, 2017). O presente trabalho adota o *framework* React Native, que utiliza JavaScript como linguagem base e compila para componentes nativos, oferecendo desempenho próximo ao de aplicações nativas.

2.1.1 Linguagem de Programação

As linguagens de programação utilizadas no desenvolvimento deste projeto dividem-se em três categorias complementares, cada uma com função específica na arquitetura da aplicação.

2.1.1.1 HTML (*HyperText Markup Language*) é a linguagem de marcação que define a estrutura e o significado semântico do conteúdo de uma página *web*. Criada por Tim Berners-Lee em 1991, evoluiu até a versão HTML5, que introduziu elementos semânticos como <header>, <nav>, <main> e <footer>, além de APIs para armazenamento local, geolocalização e renderização de gráficos (FREEMAN, 2021). No contexto do aplicativo proposto, o HTML será utilizado para estruturar as telas de login, catálogo de produtos, carrinho de compras e histórico de pedidos.

A semântica do HTML é particularmente relevante para a acessibilidade, pois permite que leitores de tela e outras tecnologias assistivas interpretem corretamente a hierarquia e a finalidade de cada elemento da interface. A *Web Content Accessibility Guidelines* (WCAG 2.1) recomenda o uso de marcação semântica como requisito fundamental para a conformidade com os padrões de acessibilidade digital (W3C, 2018).

2.1.1.2 CSS (*Cascading Style Sheets*) é a linguagem responsável pela apresentação visual dos documentos HTML. Com o CSS, é possível definir cores, fontes, espaçamentos, posicionamentos e animações, separando a aparência da estrutura do conteúdo. Essa separação é considerada uma boa prática de engenharia de *software*, pois facilita a manutenção e a evolução do sistema (DUCKETT, 2014).

No desenvolvimento do aplicativo, o CSS será empregado para criar uma interface limpa, consistente e adaptável a diferentes tamanhos de tela. Serão utilizados conceitos como **Flexbox** e **CSS Grid**, que permitem a criação de *layouts* responsivos sem a necessidade de *frameworks* externos. A escolha de uma paleta de cores contrastantes e de tamanhos de fonte adequados segue as recomendações da WCAG 2.1 para garantir a legibilidade por usuários com baixa visão.

2.1.1.3 JavaScript é a linguagem de programação que confere comportamento dinâmico às páginas *web*. Criada por Brendan Eich em 1995, evoluiu de uma linguagem simples de scripts para um ecossistema robusto com suporte a programação orientada a objetos, funcional e assíncrona (FLANAGAN, 2020).

No aplicativo proposto, o JavaScript desempenha as seguintes funções críticas:

- Gerenciamento do estado do carrinho de compras, armazenando temporariamente os itens selecionados pelo usuário;
- Validação de formulários de *login* e cadastro em tempo real, com *feedback* imediato ao usuário;
- Comunicação com o servidor por meio de chamadas assíncronas (*AJAX/Fetch API*) para envio e recuperação de dados;
- Atualização dinâmica da interface sem recarregamento completo da página, proporcionando fluidez na navegação.

Nota didática: Observe a hierarquia entre as tecnologias: HTML define o **quê** (conteúdo), CSS define o **como** (aparência) e JavaScript define o **quando** (comportamento). Essa separação de responsabilidades é um dos princípios fundamentais da engenharia de software moderna.

2.1.2 Back-end x Front-End

A arquitetura de uma aplicação *web* moderna divide-se, em termos gerais, em duas camadas complementares: o **front-end**, que corresponde à interface com a qual o usuário interage diretamente, e o **back-end**, que processa a lógica de negócio, gerencia o banco de dados e expõe serviços por meio de APIs (GARRETT, 2005).

O **front-end** do aplicativo será desenvolvido com React Native, *framework* que permite a criação de interfaces *mobile* utilizando componentes JavaScript que são renderizados como elementos nativos da plataforma (Android ou iOS). O React Native adota o paradigma de **componentização**, no qual a interface é dividida em partes reutilizáveis e independentes, cada uma com seu próprio estado e lógica de renderização (BANKS; PORCELLO, 2017).

O **back-end** será implementado como uma API REST (*Representational State Transfer*), expondo *endpoints* para operações de criação, leitura, atualização e exclusão de dados (CRUD). A API será construída utilizando Node.js com o *framework* Express, que oferece um conjunto minimalista de funcionalidades para construção de servidores *web* (POWER, 2018). Os dados serão persistidos no Firebase Firestore, serviço de banco de dados NoSQL em nuvem que oferece escalabilidade automática e integração nativa com aplicações React.

A comunicação entre *front-end* e *back-end* ocorre por meio do protocolo HTTP, com troca de dados no formato JSON (*JavaScript Object Notation*). Esse padrão é amplamente adotado na indústria por sua leveza, legibilidade e suporte universal entre linguagens de programação (RICHARDSON; RUBY, 2007).

2.1.3 Banco de Dados

O banco de dados é o componente responsável pelo armazenamento persistente e pela recuperação eficiente das informações da aplicação. Para o aplicativo de pedidos na cantina, as entidades fundamentais incluem: **usuários** (alunos com seus dados de *login* e perfil), **produtos** (itens do cardápio com preço, descrição e imagem), **pedidos** (registro de cada transação com data, itens e status) e **histórico** (registro de pedidos anteriores para consulta).

A escolha do Firebase Firestore como sistema de gerenciamento de banco de dados justifica-se por três motivos principais. Em primeiro lugar, sua natureza NoSQL oferece flexibilidade no modelo de dados, permitindo que a estrutura do banco evolua conforme os requisitos da aplicação sem a necessidade de migrações complexas de esquema (FOWLER; SADALAGE, 2012). Em segundo lugar, o Firestore oferece sincronização em tempo real, o que possibilita que o status do pedido seja atualizado automaticamente na tela do usuário sem necessidade de recarregamento manual. Em terceiro lugar, sua integração com o ecossistema Firebase simplifica a implementação de autenticação por e-mail e redes sociais, *hosting* para o *back-end* e notificações *push*.

"Bancos de dados NoSQL sacrificam a consistência imediata e a normalização relacional em favor de escalabilidade horizontal, flexibilidade de esquema e desempenho em operações de leitura e escrita em larga escala." (SADALAGE; FOWLER, 2013, p. 12)

2.2 Experiência do Usuário UX

A Experiência do Usuário (*User Experience* — UX) refere-se ao conjunto de percepções, emoções e respostas de um usuário decorrentes do uso ou da expectativa de uso de um produto, sistema ou serviço. O termo foi popularizado por Donald Norman (1990), que defendia que o design de produtos deveria considerar não apenas a funcionalidade, mas também as necessidades emocionais e cognitivas dos usuários.

No âmbito do desenvolvimento de *software*, a UX engloba aspectos como usabilidade, acessibilidade, utilidade, desejabilidade e credibilidade. Jacob Nielsen (2012) propõe que a usabilidade seja medida por cinco atributos: **facilidade de aprendizado** (o usuário consegue realizar tarefas básicas na primeira interação), **eficiência** (o usuário experiente realiza tarefas rapidamente), **memorabilidade** (o usuário retoma o uso após um período sem treinamento), **erros** (baixa taxa de erros e fácil recuperação) e **satisfação** (o usuário considera o uso agradável).

Para o aplicativo de pedidos na cantina, os princípios de UX guiarão o design das telas de forma a minimizar a carga cognitiva do usuário. A navegação será organizada em uma estrutura hierárquica simples, com no máximo três níveis de profundidade. Os botões de ação principal (como "Adicionar ao carrinho" e "Confirmar pedido") serão destacados visualmente e posicionados em áreas de fácil alcance para polegar, considerando os padrões ergonômicos de uso *mobile* com uma mão (BABICH, 2020).

A acessibilidade digital, componente essencial da UX, segue as diretrizes da **WCAG 2.1** (W3C, 2018), que estabelecem quatro princípios fundamentais: **perceptível** (as informações devem ser apresentadas de forma que todos os usuários possam percebê-las), **operável** (todos os componentes da interface devem ser operáveis por diferentes métodos de entrada), **compreensível** (a informação e a operação da interface devem ser compreensíveis) e **robusto** (o conteúdo deve ser interpretado por diferentes agentes de usuário, incluindo tecnologias assistivas).

No aplicativo proposto, a acessibilidade será contemplada por meio de: contraste mínimo de 4,5:1 entre texto e fundo, tamanho de fonte ajustável, suporte a leitores de tela com descrições textuais em imagens, áreas de toque com dimensão mínima de 44x44 pixels e navegação por teclado externo. Essas práticas beneficiam não apenas alunos com deficiência permanente, mas também aqueles com limitações situacionais, como uma lesão temporária no braço ou o uso do dispositivo sob luz solar intensa.

2.3 Framework

Um *framework* de desenvolvimento é uma abstração que fornece funcionalidades genéricas e reutilizáveis, permitindo que o desenvolvedor se concentre nas regras de negócio específicas da aplicação. Diferentemente de uma biblioteca, que o desenvolvedor chama quando necessário, o *framework* define o fluxo de controle da aplicação, seguindo o princípio de inversão de controle (GAMMA et al., 1995).

O **React Native**, criado pelo Facebook em 2015, é um *framework* de código aberto para desenvolvimento de aplicativos *mobile* que utiliza JavaScript como linguagem base. Diferentemente de abordagens puramente *web*, como *WebViews*, o React Native compila componentes React para *widgets* nativos da plataforma, resultando em desempenho superior e integração com recursos do dispositivo como câmera, geolocalização e notificações (EISENMAN, 2016).

A escolha do React Native para este projeto fundamenta-se em quatro critérios:

1. Reutilização de código: uma única base de código JavaScript pode ser compilada para Android e iOS, reduzindo o esforço de desenvolvimento e manutenção em comparação com aplicações nativas desenvolvidas separadamente para cada plataforma.

2. Hot Reload: o React Native permite que alterações no código sejam visualizadas em tempo real no dispositivo ou emulador, sem necessidade de recompilação completa, acelerando significativamente o ciclo de desenvolvimento e teste.

3. Comunidade ativa: com mais de 100 mil estrelas no GitHub e uma vasta coleção de bibliotecas de terceiros, o ecossistema React Native oferece soluções prontas para funcionalidades comuns como navegação entre telas, armazenamento local e autenticação.

4. Curva de aprendizado: para desenvolvedores com conhecimento prévio de JavaScript e React (para *web*), a transição para React Native é natural, pois a API e o paradigma de componentes são essencialmente os mesmos.

Outros *frameworks* foram considerados durante a fase de planejamento. O **Flutter**, da Google, utiliza a linguagem Dart e oferece desempenho nativo com um sistema de renderização próprio. O **Ionic**, baseado em *WebViews*, permite o uso de tecnologias *web* tradicionais, mas apresenta desempenho inferior em animações complexas. O **Xamarin**, da Microsoft, utiliza C# e oferece integração com o ecossistema .NET. A opção pelo React Native reflete o equilíbrio entre desempenho, produtividade e alinhamento com as competências desenvolvidas ao longo do curso técnico.

2.4 Propriedade de Software

O regime de propriedade intelectual de *software* estabelece os direitos e deveres de desenvolvedores e usuários em relação ao código-fonte, à documentação e ao produto final. No Brasil, o *software* é protegido pela **Lei nº 9.609/1998** (Lei do *Software*), que equipara os programas de computador às obras literárias para fins de proteção autoral, e pela **Lei nº 9.610/1998** (Lei de Direitos Autorais).

Do ponto de vista da distribuição e do licenciamento, os *softwares* classificam-se em duas grandes categorias. O ***software proprietário*** é aquele cujo código-fonte não é disponibilizado publicamente, sendo seu uso condicionado à aquisição de uma licença que impõe restrições quanto à cópia, modificação e redistribuição. Exemplos incluem o Microsoft Windows, o Adobe Photoshop e o Oracle Database. O ***software de código aberto*** (*open source*), por sua vez, disponibiliza o código-fonte publicamente, permitindo que qualquer pessoa estude, modifique e distribua o programa, desde que respeite os termos da licença específica (STALLMAN, 2002).

O desenvolvimento do aplicativo proposto será baseado integralmente em tecnologias de código aberto: JavaScript, Node.js, React Native, Firebase e Figma (plano educacional gratuito). As licenças associadas a essas ferramentas (MIT, Apache 2.0, BSD) permitem o uso comercial e acadêmico sem custos de licenciamento, o que é particularmente relevante para instituições públicas de ensino.

O código-fonte do aplicativo será disponibilizado sob a licença **MIT**, que permite a qualquer pessoa usar, copiar, modificar, fundir, publicar e distribuir o *software* sem restrições, desde que o aviso de direito autoral seja mantido. Essa escolha visa incentivar a colaboração acadêmica, permitindo que outros alunos e instituições adaptem a solução para suas realidades específicas.

Nota didática: A escolha da licença de software não é uma decisão meramente técnica. Ela reflete valores éticos (cultura de compartilhamento vs. proteção comercial), impacta o potencial de reuso e pode influenciar a adoção da solução por terceiros. Para projetos acadêmicos, licenças permissivas como MIT ou Apache 2.0 são geralmente as mais adequadas.

3. Projeto

Esta seção apresenta o projeto técnico do aplicativo, detalhando a análise de requisitos, as telas que compõem a interface e a arquitetura geral do sistema. O projeto segue as boas práticas de engenharia de *software* estabelecidas por Pressman (2016) e Sommerville (2018), com ênfase na documentação clara e na rastreabilidade entre requisitos e funcionalidades implementadas.

3.1 Projeto Técnico

O projeto técnico constitui a especificação detalhada do sistema, traduzindo os requisitos levantados na fase de análise em uma solução concreta. Esta subseção organiza-se em três partes: a análise de requisitos, a descrição das telas e a definição da arquitetura.

3.1.1 Análise de Requisitos

A análise de requisitos é uma etapa crítica do ciclo de vida do *software*, pois erros de especificação nessa fase são os mais custosos para corrigir em etapas posteriores (SOMMERVILLE, 2018). Os requisitos foram levantados por meio de questionários aplicados com alunos da FATEC Arthur de Azevedo e de entrevistas informais com a equipe da cantina. A seguir, apresentam-se os requisitos funcionais e não funcionais identificados.

Requisitos Funcionais (RF)

ID	Descrição	Prioridade
RF01	O sistema deve permitir que o usuário realize <i>login</i> com e-mail e senha	Alta
RF02	O sistema deve exibir o catálogo de produtos disponíveis com nome, descrição, preço e imagem	Alta
RF03	O sistema deve permitir que o usuário adicione produtos a um carrinho de compras	Alta
RF04	O sistema deve permitir que o usuário visualize, edite a quantidade e remova itens do carrinho	Alta
RF05	O sistema deve exibir o valor total do carrinho, atualizado em tempo real	Alta
RF06	O sistema deve permitir que o usuário confirme o pedido, escolhendo retirada no balcão ou entrega em ponto combinado	Alta
RF07	O sistema deve enviar uma notificação ao usuário quando o pedido estiver pronto para retirada	Média
RF08	O sistema deve exibir o histórico de pedidos realizados pelo usuário	Média
RF09	O sistema deve permitir que o usuário edite seu perfil (nome, foto, preferências)	Baixa

RF10	O sistema deve autenticar o usuário por meio de token JWT (<i>JSON Web Token</i>)	Alta
------	---	------

Requisitos Não Funcionais (RNF)

ID	Descrição	Categoria
RNF01	O sistema deve criptografar senhas utilizando <i>hash</i> bcrypt	Segurança
RNF02	O tempo de resposta das requisições à API não deve exceder 3 segundos em conexão 4G	Performance
RNF03	A interface deve ser operável com uma mão, com botões posicionados na metade inferior da tela	Usabilidade
RNF04	O aplicativo deve ser compatível com Android 8.0+ e iOS 13+	Compatibilidade
RNF05	O contraste mínimo entre texto e fundo deve ser de 4,5:1 (padrão WCAG AA)	Acessibilidade
RNF06	O aplicativo deve funcionar em modo <i>offline</i> parcial, com cache do catálogo de produtos	Disponibilidade

Nota didática: Observe que os requisitos funcionais descrevem **o que** o sistema deve fazer, enquanto os não funcionais descrevem **como** o sistema deve se comportar. Ambos são igualmente importantes: um sistema que funciona mas é inseguro ou inacessível não atende plenamente às necessidades dos usuários.

3.1.2 Telas do Projeto

A interface do aplicativo é composta por seis telas principais, projetadas seguindo os princípios de UX discutidos na revisão bibliográfica. A navegação entre as telas será gerenciada pelo React Navigation, biblioteca que implementa o padrão de **pilha de navegação** (*stack navigator*), permitindo que o usuário avance e retorne entre as telas de forma intuitiva.

1. Tela de Login (T1): apresenta campos para inserção de e-mail e senha, botão "Entrar" e link para "Cadastre-se" para novos usuários. A tela exibe o logotipo da cantina e um breve texto de boas-vindas. Em caso de erro de autenticação, mensagens claras são exibidas. O *design* segue o princípio de *affordance*, com campos estilizados como retângulos com bordas arredondadas que sugerem sua função de entrada de dados (NORMAN, 1990).

2. Tela de Cadastro (T2): formulário com campos para nome completo, e-mail, senha e confirmação de senha. A validação é feita em tempo real: o campo de e-mail verifica o formato, o campo de senha exige mínimo de 6 caracteres com pelo menos uma letra e um número, e a confirmação compara com a senha digitada. Mensagens de erro específicas orientam o usuário na correção.

3. Tela do Catálogo (T3): exibe os produtos organizados em categorias (lanches, bebidas, sobremesas, salgados). Cada produto é representado por um cartão com imagem, nome, descrição resumida e preço. O usuário pode tocar em um produto para ver detalhes ou adicioná-lo diretamente ao carrinho. Um botão flutuante no canto inferior direito exibe o número de itens no carrinho e leva à tela de carrinho. A tela suporta busca por texto e filtro por categoria.

4. Tela do Carrinho (T4): lista todos os itens adicionados, com nome, quantidade (ajustável por botões "+" e "-"), preço unitário e subtotal. O valor total é exibido no rodapé, junto com o botão "Confirmar Pedido". O usuário pode remover itens individualmente com um *swipe* lateral ou botão de exclusão. Antes da confirmação, o sistema solicita que o usuário escolha entre "Retirar no balcão" ou "Entrega em ponto combinado", exibindo um mapa simplificado com os pontos de entrega disponíveis no campus.

5. Tela de Confirmação (T5): exibe um resumo do pedido com número do protocolo, itens, valor total, forma de retirada/entrega escolhida e status atual ("Recebido", "Em preparo", "Pronto"). A tela é atualizada em tempo real via Firebase Firestore, permitindo que o usuário acompanhe o progresso sem sair do aplicativo. Um botão "Ver Histórico" leva à tela seguinte.

6. Tela de Histórico (T6): lista os pedidos anteriores do usuário ordenados por data (do mais recente para o mais antigo). Cada pedido exibe data, valor total, número de itens e status final. O usuário pode tocar em um pedido para ver os detalhes completos. Essa tela permite que o aluno refaça pedidos frequentes com apenas dois toques, economizando tempo em compras futuras.

A tela de Perfil, com prioridade baixa, será implementada em versão futura e permitirá edição de nome, alteração de senha, configuração de preferências alimentares e exclusão da conta.

3.1.3 Arquitetura

A arquitetura do sistema segue o modelo **cliente-servidor**, com separação clara entre a camada de apresentação (*front-end* mobile), a camada de lógica de negócio (*back-end* em API REST) e a camada de dados (Firebase Firestore). Essa separação permite que cada camada evolua de forma independente, facilitando a manutenção e a escalabilidade do sistema (PRESSMAN, 2016).

O **front-end**, rodando no dispositivo móvel do usuário (Android ou iOS), é responsável por:

- Renderizar a interface do usuário com base no estado atual da aplicação;
- Capturar eventos de toque e entrada de dados;
- Gerenciar o estado local do carrinho de compras enquanto o usuário navega;
- Realizar requisições HTTP para a API REST para operações que exigem persistência ou consulta a dados;
- Armazenar em cache local (*AsyncStorage*) o catálogo de produtos para funcionamento *offline* parcial.

O **back-end**, hospedado no Firebase Cloud Functions ou em um servidor Node.js com Express, é responsável por:

- Autenticar usuários por meio de e-mail/senha ou Google OAuth;
- Processar a lógica de negócio para criação e atualização de pedidos;
- Validar dados enviados pelo *front-end* antes de persistir no banco de dados;
- Notificar o *front-end* sobre mudanças de status dos pedidos via *WebSocket* ou *listeners* do Firestore.

O **Firebase Firestore** armazena os dados em coleções e documentos, organizados da seguinte forma:

usuarios/ { id, nome, email, senha_hash, foto_url, preferencias_alimentares, created_at }

produtos/ { id, nome, descricao, preco, categoria, imagem_url, disponivel, created_at }

pedidos/ { id, usuario_id, itens: [{ produto_id, quantidade, preco_unitario }], total, forma_entrega, ponto_entrega, status, created_at, updated_at }

A arquitetura contempla ainda um sistema de **fila de pedidos** no *back-end*, que garante que pedidos concorrentes sejam processados na ordem correta, evitando conflitos de atualização no banco de dados. O diagrama a seguir ilustra o fluxo completo de uma requisição típica:

Usuário → [Tela do Catálogo] → seleciona produto → adiciona ao carrinho (estado local) → confirma pedido → requisição POST para API → API valida dados → persiste no Firestore → retorna confirmação com protocolo → [Tela

de Confirmação] exibe status → *listener* Firestore notifica mudanças de status → [Tela de Confirmação] atualiza em tempo real.

Nota didática: A escolha de uma arquitetura cliente-servidor com API REST é o padrão mais utilizado na indústria atual. Ela permite que o mesmo back-end atenda a diferentes front-ends (mobile, web, smart TVs), maximizando o reuso de código e simplificando a manutenção do sistema.

4. Considerações Finais

O presente trabalho propôs o desenvolvimento de um aplicativo *mobile* para pedidos na cantina da FATEC Arthur de Azevedo, com o objetivo central de promover a acessibilidade para alunos com mobilidade reduzida. O problema de pesquisa partiu da constatação de que a estrutura física do campus impõe barreiras significativas de locomoção, que a tecnologia pode ajudar a mitigar por meio da antecipação digital dos pedidos.

Os resultados esperados com a implementação da solução são: (a) redução do tempo e do esforço físico despendidos por alunos com mobilidade reduzida para acessar os serviços da cantina; (b) aumento da autonomia desses alunos, que poderão realizar pedidos sem depender de terceiros para o deslocamento; (c) melhoria na eficiência operacional da cantina, que poderá preparar os pedidos com antecedência, reduzindo filas e tempo de espera; e (d) criação de um modelo replicável para outras instituições de ensino que enfrentem desafios semelhantes de acessibilidade.

A metodologia adotada — pesquisa aplicada com etapas de levantamento bibliográfico, pesquisa de campo, análise de requisitos, prototipação, desenvolvimento e testes — mostrou-se adequada para o escopo do trabalho, permitindo que a solução fosse construída com base em dados reais sobre as necessidades dos usuários e em fundamentos teóricos consolidados da engenharia de *software*, da experiência do usuário e da acessibilidade digital.

É importante, no entanto, reconhecer as **limitações** do presente estudo. Em primeiro lugar, a implementação do pagamento digital por meio de cartão de crédito, débito ou *Pix* não foi contemplada nesta versão do aplicativo, limitando-se à confirmação do pedido com pagamento a ser realizado presencialmente. Essa funcionalidade demandaria a integração com *gateways* de pagamento como Stripe, PagSeguro ou Mercado Pago, o que envolve requisitos de segurança adicionais (PCI DSS) e custos de transação que fogem ao escopo de um trabalho de conclusão de curso técnico.

Em segundo lugar, a logística de entrega em pontos combinados dentro do campus requer a definição precisa de locais de encontro e a coordenação entre o horário de preparo do pedido e o deslocamento do aluno. Em horários de pico, a alta demanda pode sobrecarregar a cantina e comprometer os prazos estimados. Esses desafios operacionais precisarão ser endereçados em conjunto com a administração do campus e com a equipe da cantina.

Em terceiro lugar, os testes de usabilidade foram realizados com uma amostra não probabilística de alunos voluntários, o que limita a generalização dos resultados para toda a população acadêmica. Estudos futuros poderão ampliar a amostra e incluir alunos com diferentes tipos de deficiência para avaliar a eficácia do aplicativo em um espectro mais amplo de necessidades de acessibilidade.

Como **trabalhos futuros**, sugere-se:

- Implementação de pagamento digital integrado, com suporte a múltiplos meios de pagamento e *cashback* institucional;
- Desenvolvimento de um painel gerencial (*dashboard*) para a cantina, com métricas de vendas, produtos mais pedidos, horários de pico e desempenho operacional;
- Integração com sistemas acadêmicos existentes (como o SIGA) para autenticação unificada e vinculação de pedidos ao cartão de estudante;
- Expansão do aplicativo para outros serviços do campus, como agendamento de biblioteca, reserva de salas de estudo e compra de ingressos para eventos;

— Realização de um estudo longitudinal para avaliar o impacto do aplicativo na frequência de uso da cantina por alunos com mobilidade reduzida ao longo de um semestre letivo.

Conclui-se que o aplicativo proposto representa uma contribuição concreta para a democratização do acesso aos serviços do campus, alinhando-se aos princípios de inclusão, autonomia e dignidade que devem nortear as instituições de ensino. A tecnologia, quando projetada com foco nas necessidades reais dos usuários, pode ser um poderoso instrumento de transformação social — e é nessa direção que este trabalho espera apontar.

Nota: este documento foi elaborado como trabalho de conclusão do curso técnico em informática da FATEC Arthur de Azevedo. Os nomes de produtos e serviços mencionados são marcas registradas de seus respectivos proprietários.

Referências Bibliográficas

BABICH, N. **UX Design for Mobile: A Beginner's Guide to Creating a Great User Experience for Mobile Apps**. Berkeley: Apress, 2020.

BANKS, A.; PORCELLO, E. **Learning React: Functional Web Development with React and Redux**. Sebastopol: O'Reilly Media, 2017.

DUCKETT, J. **HTML and CSS: Design and Build Websites**. Indianapolis: John Wiley & Sons, 2014.

EISENMAN, B. **Learning React Native: Building Native Mobile Apps with JavaScript**. Sebastopol: O'Reilly Media, 2016.

FLANAGAN, D. **The Definitive Guide**. 7. ed. Sebastopol: O'Reilly Media, 2020.

FOWLER, M.; SADALAGE, P. J. **NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence**. Boston: Addison-Wesley, 2012.

FREEMAN, E. **Head First HTML and CSS**. 2. ed. Sebastopol: O'Reilly Media, 2021.

GAMMA, E. et al. **Design Patterns: Elements of Reusable Object-Oriented Software**. Boston: Addison-Wesley, 1995.

GARRETT, J. J. **The Elements of User Experience: User-Centered Design for the Web and Beyond**. Berkeley: New Riders, 2005.

INEP — Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira. **Censo da Educação Superior 2023: notas estatísticas**. Brasília: INEP, 2023.

MARCOTTE, E. **Responsive Web Design**. New York: A Book Apart, 2011.

NIELSEN, J. **Usability Engineering**. San Francisco: Morgan Kaufmann, 2012.

NORMAN, D. **The Design of Everyday Things**. New York: Basic Books, 1990.

PNAD — Pesquisa Nacional por Amostra de Domicílios. **Acesso à Internet e à Televisão e Posse de Telefone Móvel Celular para Uso Pessoal 2022**. Rio de Janeiro: IBGE, 2022.

POWER, K. **Express in Action: Writing, Building, and Testing Node.js Applications**. Shelter Island: Manning Publications, 2018.

PRESSMAN, R. S. **Engenharia de Software: Uma Abordagem Profissional**. 8. ed. Porto Alegre: AMGH, 2016.

RICHARDSON, L.; RUBY, S. **RESTful Web Services**. Sebastopol: O'Reilly Media, 2007.

SADALAGE, P. J.; FOWLER, M. **NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence**. Boston: Addison-Wesley, 2013.

SHEPPARD, D. **Beginning Progressive Web App Development: Creating a Native App Experience on the Web**. Berkeley: Apress, 2017.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2018.

STALLMAN, R. **Free Software, Free Society: Selected Essays of Richard M. Stallman**. Boston: GNU Press, 2002.

W3C — World Wide Web Consortium. **Web Content Accessibility Guidelines (WCAG) 2.1**. Cambridge: W3C, 2018.

WAZLAWICK, R. S. **Metodologia de Pesquisa para Ciência da Computação**. 2. ed. Rio de Janeiro: Elsevier, 2014.